NR2DT-00: Tenant Prerequisites — Minimum Setup Before Migration

> **Series:** NR2DT — New Relic to Dynatrace Migration Steps | **Reference:** 00 — Tenant Prerequisites | **Created:** May 2026 | **Last Updated:** 10/25/2026

Overview

Goal of this step: confirm the target Dynatrace tenant is in a fit state to receive a New Relic migration. This is the readiness gate before NR2DT-01 (Discover) begins.

NR2DT-00 is intentionally consultative, not procedural. NR2DT-03 (Design) is where the bucket / IAM / OpenPipeline architecture is *designed* and committed to Terraform. NR2DT-05 Wave 0 is where it is *applied*. This notebook lists the items that must exist (or must be planned and signed off) in the target tenant before any of that begins, and points to the deeper series that explain each one.

If a row in the readiness checklist (§5) cannot be ticked, stop and resolve it before opening NR2DT-01.

Table of Contents

1. Tenant Foundation (Mandatory)
2. Access & Tooling (Mandatory)
3. Telemetry Infrastructure (Mandatory)
4. Operational Baseline (Strongly Recommended)
5. Readiness Checklist
6. Where Each Topic Is Covered

Prerequisites

Requirement	Details
Audience	Migration lead + Dynatrace tenant owner + IAM admin
Format	Readiness gate — items only; deeper *how* lives in the linked series
Output	A signed-off readiness checklist; no migration artifacts produced here
Outcome	NR2DT-01 can begin

1. Tenant Foundation (Mandatory)

These five items shape the target Dynatrace tenant. They do not all have to be **applied** before NR2DT-01 (that happens in NR2DT-05 Wave 0), but they must be **planned and signed off** before discovery starts. NR2DT-01 surfaces NR-side decisions that depend on knowing the target shape — bucket names, host group taxonomy, security context values.

1.1 Bucket Strategy

Recommended Approach: decide bucket naming, retention, and pricing model before any logs flow. Use a single `<org>_` prefix so IAM policies can scope with one `storage:bucket-name startsWith "<org>_"` condition.

Consideration: buckets are immutable once created — names cannot be changed and retention can only be adjusted within limits. There is also a per-environment default limit of 250 custom buckets from SaaS 1.346 (staged tenant rollout from 08/25/2026; 80 before it) — *"The default limit per environment is 250 custom buckets."* (How to organize your data stored in Grail (DT docs)); design with that ceiling in mind. Buckets are a scenario-driven mechanism (compliance retention, hard cost partitioning, hostile multi-tenancy) — not the general data-access mechanism. For general access scoping, prefer `dt.security_context` (§1.3).

Trade-offs of Deferring: logs land in the `default_logs` bucket; later re-routing breaks dashboards that filter on bucket and forces a re-validation pass.

Where covered:
- ORGNZ-02 — Understanding Grail Buckets
- ORGNZ-99 — Best Practice Summary & DQL Reference

1.2 Host Group Taxonomy

Recommended Approach: group hosts by ownership × environment (e.g., `prod-app-payments`, `nonprod-app-payments`). Set the host group at OneAgent install time via `oneagentctl --set-host-group`.

Consideration: retroactively re-grouping hosts breaks history-based thresholds, IAM scoping, and any automation keyed on host group. Sprint 1.337 lets OneAgent emit `team`, `cost_center`, `env` as primary tags directly via `oneagentctl --set-host-tag` — prefer this over OpenPipeline post-processing where possible.

Trade-offs of Deferring: a single all-hosts default group breaks per-environment alerting, blocks IAM scoping, and forces a post-migration re-tagging sweep.

Where covered:
- FAQ-01 — Host Group Naming Strategy
- ONBRD series — Host group setup at OneAgent install time

1.3 `dt.security_context` Tagging Strategy

Recommended Approach: decide the `dt.security_context` value space before tagging anything. `dt.security_context` is the universal field that Gen3 IAM policies use to scope access across both configurations and data — without it, cross-entity-type policies cannot be written.

Consideration: `dt.security_context` is the default mechanism for general data access scoping. Bucket-based scoping is reserved for the specific scenarios noted in §1.1.

Trade-offs of Deferring: every IAM binding that lands without a `dt.security_context` filter becomes an unscoped policy that grants broader access than intended.

Where covered:
- FAQ-02 — Tagging Sources, Standards, and Strategy
- ORGNZ-06 — Security Context for Data Access
- ORGNZ-07 — Security Context for Configuration Scoping

1.4 IAM Parameterized Policies

Recommended Approach: prefer one parameterized policy with binding parameters over many copies of the same policy with hardcoded scope values. Bind the policy to multiple groups, varying only the parameter (e.g., `team_name`, `environment`).

Consideration: the policy parameter shape is load-bearing. Changing it later cascades across every binding that uses the policy — design once, change rarely.

Trade-offs of Deferring: importing dashboards/notebooks before IAM is ready means artifacts land with no access model — visible to admins only, invisible to the teams that need them.

Where covered:
- IAM-04 — Designing Effective Policies
- IAM-05 — Boundary Conditions and Scoping
- IAM-11 — WORKSHOP: Policy & Persona Design
- IAM-99 — Best Practice Summary & DQL Reference

1.5 OpenPipeline Routing & Enrichment

Recommended Approach: define routing rules (first-match-wins), enrichment fields (`dt.security_context`, `environment`, `team`), and drop rules before the first NR-replacement log forwarder points at the tenant.

Consideration: OpenPipeline is the equivalent of NR's tag/workload model. Logs arriving without enrichment land without a `dt.security_context`, which means IAM scoping will not work on them — and reprocessing later is expensive.

Trade-offs of Deferring: logs accumulate without routing — they hit the default bucket, retain at default settings, and have to be re-processed once enrichment is added.

Where covered:
- OPLOGS series — OpenPipeline log processing
- OPMIG series — Classic Logs → OpenPipeline migration
- OPIPE series — OpenPipeline beyond logs (spans, metrics, events)

2. Access & Tooling (Mandatory)

2.1 Platform Tokens & Scopes

Recommended Approach: issue a single Platform Token for the migration tooling with the scope set listed in NR2DT REFERENCE.md § Required API Tokens. Platform Tokens are the recommended default; Classic API tokens are being phased out; OAuth clients are reserved for external system integrations or account-admin automation.

Consideration: missing scopes only surface mid-migration when a specific component fails to write. The `migrate.py preflight` check (§2.2) catches the major ones up front.

```
export DYNATRACE_API_TOKEN="dt0s16....."  # Platform Token
export DYNATRACE_ENVIRONMENT_URL="https://&lt;env-id&gt;.live.dynatrace.com"
```

Where covered:
- NR2DT-01 § Prerequisites & Access (full scope list)
- IAM series — Token management lessons

2.2 Run `migrate.py preflight`

Recommended Approach: run preflight before any inventory work begins. It probes the tenant for Settings 2.0, Document API, and Automation API availability, and reports `scopes_min` / `scopes_recommended` / `diagnosis` / `remediation` per check.

```
python3 migrate.py preflight
```

Consideration: preflight uses the same token as the main run. A clean preflight is the single best signal that the tenant is fit to migrate to.

Trade-offs of Deferring: the failure surfaces inside Wave 0 instead of before it, costing one wave's worth of rework.

Where covered: NR2DT-01 § Prerequisites & Access

2.3 Service User for Migrated Anomaly Detectors

Recommended Approach: create a service user for the migrated Davis anomaly detectors and set `DYNATRACE_DETECTOR_ACTOR` to its UUID in the tool's `.env` before NR2DT-05 Wave 3. Every custom alert executes as an actor, and Dynatrace recommends a service user rather than a person for alerts a team owns: *"We recommend using service users as actors for custom alerts created for a department or organization use case."*

```
export DYNATRACE_DETECTOR_ACTOR="&lt;service-user-uuid&gt;"
```

Consideration: `migrate.py` imports NR alert conditions as Davis anomaly detectors, whose `executionSettings.actor` must be a service-user UUID. When the variable is

unset, the `migrate` command (not `preflight`) prints a warning and the detectors fail to import, so Wave 3 fails on its first run.

Where covered: NR2DT-05 § Wave 3

> **Sources:** Anomaly Detection app (DT docs), migrate.py @ 78cbfce (tool repo, GitHub), config/settings.py @ 78cbfce (tool repo, GitHub), .env.example @ 78cbfce (tool repo, GitHub), read 10/05/2026.

3. Telemetry Infrastructure (Mandatory)

3.1 OneAgent / ActiveGate Rollout

Recommended Approach: roll OneAgent to the hosts that will be in scope before NR2DT-08 Validate runs. ActiveGate sits in front for log routing, OTel reception, and CloudWatch / Azure Monitor / GCP integrations. For NR→DT migrations specifically, decide the OneAgent/OTel split per workload before instrumenting.

Consideration: OneAgent is the default for JVM, .NET, Node, Go, Python — code-free, infra correlation, host-level metrics. OTel is the choice for effect-system languages (Cats Effect, ZIO), polyglot meshes, or workloads where instrumentation-as-code is a hard requirement. Capture the per-workload decision in writing before instrumenting; the OTEL series covers the underlying capabilities and trade-offs.

Where covered:
- ONBRD series — Dynatrace onboarding fundamentals
- OTEL series — OpenTelemetry integration and the OneAgent/OTel decision space

---

3.2 Log Forwarder Endpoints

Recommended Approach: know the Dynatrace log ingest URL and authentication method before NR2DT-07 reconfigures the log shippers (Fluent Bit, Fluentd, Vector, etc.). Confirm the ActiveGate hostname, port, and routing rules.

Consideration: the cutover from NR log destinations to DT happens in NR2DT-07 Wave 5. Late discovery of a missing ActiveGate or wrong endpoint blocks the wave.

Where covered:
- OPLOGS series — log ingestion and processing
- OPMIG series — Classic Logs → OpenPipeline

3.3 OTel Collector Endpoints (if applicable)

Recommended Approach: if any source-side workload emits OTLP rather than going through OneAgent, confirm the Dynatrace OTLP endpoint is configured and reachable, and that the OTel collector is forwarding to it with the correct API token.

Consideration: Sprint 1.337's OneAgent + OTel `service.name` enrichment means OTel-instrumented services map cleanly to DT services without the historical two-services-per-process pattern. This affects taxonomy decisions in NR2DT-03.

Where covered:
- OTEL series — OpenTelemetry integration
- NRLC-01 § Sprint 1.337 — OneAgent + OTel mapping

4. Operational Baseline (Strongly Recommended)

These items are not blockers for NR2DT-01, but every migration that has skipped them has had to revisit them mid-wave.

4.1 Silent Notification Channels

Recommended Approach: create a silent notification channel (Slack channel, email DL, webhook to /dev/null) for the dual-alert window in NR2DT-05 Wave 3. Dynatrace alerts fire to this channel for 1–2 weeks alongside live NR alerts so on-call can compare without being paged twice.

Consideration: the dual-alert window is non-negotiable per NR2DT REFERENCE.md Lessons Learned. Skipping it has caused on-call surprises in every migration that tried.

Where covered: WFLOW series — Workflows and Alert Notifications

---

4.2 Davis Sensitivity Defaults

Recommended Approach: review Davis adaptive baseline sensitivity defaults and problem suppression windows for the tenant. NR APM conditions become Dynatrace adaptive baselines — fundamentally different math, fundamentally different defaults.

Consideration: APM conditions do not auto-translate. Plan manual review of every NR APM condition against the Dynatrace Intelligence baseline behavior before NR2DT-05 Wave 3.

Where covered:
- AIOPS series — Dynatrace Intelligence (anomaly detection, Davis problems & RCA)
- NRLC-04 — Alert & Workflow deep dive

5. Readiness Checklist

NR2DT-01 cannot start until every Mandatory item is ticked. Strongly Recommended items can be deferred but the deferral must be deliberate and recorded.

Mandatory

- [ ] Bucket naming, retention, and pricing model decided (§1.1)
- [ ] Host group taxonomy decided; OneAgent install procedure includes `--set-host-group` (§1.2)
- [ ] `dt.security_context` value space decided (§1.3)
- [ ] IAM parameterized policy shape decided; group → policy bindings drafted (§1.4)
- [ ] OpenPipeline routing + enrichment + drop rules drafted (§1.5)
- [ ] Platform Token issued with required scopes (§2.1)
- [ ] `migrate.py preflight` returns clean (§2.2)
- [ ] Service user created for anomaly detectors; `DYNATRACE_DETECTOR_ACTOR` set (§2.3)
- [ ] OneAgent / ActiveGate rollout plan in place; OneAgent/OTel split decided per workload (§3.1)
- [ ] Log forwarder destination endpoints known (§3.2)
- [ ] OTel collector endpoints confirmed if applicable (§3.3)

Strongly Recommended

- [ ] Silent notification channel created for dual-alert window (§4.1)
- [ ] Davis sensitivity defaults reviewed (§4.2)

Verification Queries (after Wave 0)

Once Wave 0 is applied (NR2DT-05), use these DQL queries to confirm the foundation is in place. Run them before declaring W0 (Wave 0 gate, NR2DT-02 §4) complete.

Verify hosts are reporting (smoke test for OneAgent rollout):

```
// Active hosts reporting CPU metrics in the last hour
timeseries cpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.smartscape.host}
| summarize active_hosts = count()
```

Verify `dt.security_context` is populated on logs:

```
// Distribution of security_context values across recently ingested logs
fetch logs, from:-1h
| summarize log_count = count(), by:{dt.security_context}
| sort log_count desc
| limit 20
```

A row of `dt.security_context = null` above a small percentage of total log volume means OpenPipeline enrichment is missing a case — fix before declaring W0 complete.

Verify the expected bucket is receiving logs (run once per expected bucket):

```
// Smoke test: confirm the named bucket has recent log volume
fetch logs, from:-15m, scanLimitGBytes:1
| filter dt.system.bucket == "&lt;org&gt;_app_logs"
| summarize log_count = count()
```

Swap the bucket name and re-run for each expected bucket. A `log_count` of 0 means the routing rule for that bucket is not matching — review the OpenPipeline configuration. (The query always returns one row, so check the value, not the row count.)

6. Where Each Topic Is Covered

Quick lookup if a stakeholder asks *where do I learn more?*

Topic	Primary Series	Supporting References
Bucket strategy	ORGNZ-02	ORGNZ-99
Host group taxonomy	FAQ-01	ONBRD
`dt.security_context` tagging	FAQ-02	ORGNZ-06, ORGNZ-07
IAM parameterized policies	IAM-11 (WORKSHOP)	IAM-04, IAM-05, IAM-99
OpenPipeline routing & enrichment	OPLOGS, OPMIG	OPIPE
Platform tokens & scopes	NR2DT-01 § Access	IAM series
OneAgent rollout	ONBRD	—
OneAgent and OTel decision	OTEL series	ONBRD
Log forwarder migration	OPMIG, OPLOGS	NR2DT-07
OTel collector setup	OTEL	NRLC-01 § Sprint 1.337
Silent notification channel	WFLOW	NR2DT-05 Wave 3
Davis sensitivity defaults	AIOPS	NRLC-04

For the consolidated NR2DT runbook view: NR2DT-99 Best Practice Summary.

Next step: NR2DT-01 — Discover (NR-side inventory + translation report + effort estimate).

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine, and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.